ZENIKA · SPEED TRAINING IA · 4H

Développeur IA — Avancé

Un dev, plusieurs agents, un système

Configurer, isoler, juger, paralléliser : passer d'un agent à plusieurs.

Agenda

0 Le pont — 10 min

1 L’économie de l’agent — 20 min

2 Le harness, actif d’équipe — 45 min

3 Ingénierie du contexte — 25 min, *puis pause 10 min*

4 Spec-driven et qualité — 45 min

5 Patterns agentiques — 30 min

6 Paralléliser : worktrees et ADE — 40 min, *puis clôture 10 min*

© ZENIKA 2026 | All rights reserved

Module 0 · 10 min

Le pont

On reprend là où la Foundation s’arrête.



© ZENIKA 2026 | All rights reserved

Quiz de positionnement

QUIZ 3 min

1

« npm test avant chaque commit » : instructions projet ou demande initiale ?

Instructions projet

2

Une consigne qu'on répète trois fois devient… ?

Une skill

3

Skill ou hook : lequel s'exécute à coup sûr ?

Le hook

4

Deux agents en même temps, un seul dépôt : que faut-il à chacun ?

Sa branche et son répertoire

© ZENIKA 2026 | All rights reserved

Reprendre au cran 4

Niveau d’autonomie	Ce qu’il demande	Où
Observer sans agir	une demande cadrée, un plan relu	Foundation
Demander à chaque action	savoir lire ce que l’agent propose	Foundation
Agir dans le périmètre	tests, un commit par étape, diff relu	Foundation
Aller au bout seul	un harness configuré, un environnement isolé, un juge	aujourd’hui

Aujourd’hui : **configurer** (modules 1 à 3), **vérifier** (4), **juger** (5), **isoler et paralléliser** (6).

© ZENIKA 2026 | All rights reserved

Module 1 · 20 min

L’économie de l’agent

Où part la facture d’un agent, et comment la réduire ?



© ZENIKA 2026 | All rights reserved

Qui parle à qui : la pile complète

Méthode : SDD spec.md → plan.md → tasks.md, relus par l'humain

Orchestrateur découpe, distribue, attend, agrège, reprend sur erreur

Agent A	Agent B	Agent C
harness : boucle, tools, contexte	harness : boucle, tools, contexte	harness : boucle, tools, contexte
appel au modèle	appel au modèle	appel au modèle

Gateway + guardrails auth, routage, quotas, cache, journalisation, filtrage entrée / sortie

LLM cloud	Hyperscaler	Souverain	SLM local
Anthropic, OpenAI...	*Bedrock, Vertex, Azure*	*Mistral, Scaleway...*	*Ollama, vLLM*

L'humain écrit et valide la spec, puis supervise aux points clés.

Outils

fichiers, terminal, git, navigateur

serveurs MCP : GitHub, Jira, BDD...

exécutés par le harness, jamais par le modèle

*Point de passage obligé : c'est là que l'entreprise **contrôle**.*

© ZENIKA 2026 | All rights reserved

Harness vs orchestrateur : la frontière

HARNESS

Fait d'un modèle un agent

- Gère une boucle d'agent
- Exécute les tools, applique les permissions
- Construit et compacte le contexte

Claude Code | Codex CLI | Copilot CLI | OpenCode

ORCHESTRATEUR

Fait travailler plusieurs agents

- Décompose et route les tâches
- Parallélise, attend, agrège
- Persiste l'état, reprend après panne

LangGraph | CrewAI | OpenAI Agents SDK | Temporal

La frontière bouge : retenez le **rôle**, pas le produit.

© ZENIKA 2026 | All rights reserved

Du KV cache au prompt caching

LE PRINCIPE

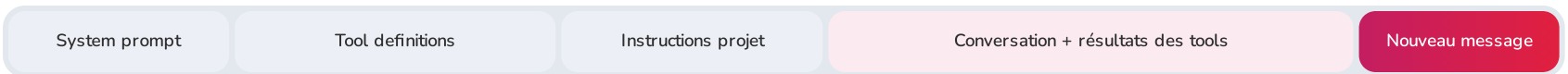
Le fournisseur garde le KV cache d'un préfixe déjà vu

Même début de requête : le prefill est sauté. Plus rapide, **beaucoup moins cher**.

~90 %

de remise sur l'entrée lue en cache

ANATOMIE D'UNE REQUÊTE D'AGENT : DU PLUS STABLE AU PLUS VOLATIL



← *stable : servi par le cache d'un tour à l'autre*

volatil : seule la fin est recalculée →

CACHE EXPIRÉ

L'agent n'a **rien oublié** : la conversation est renvoyée en entier.

MAIS

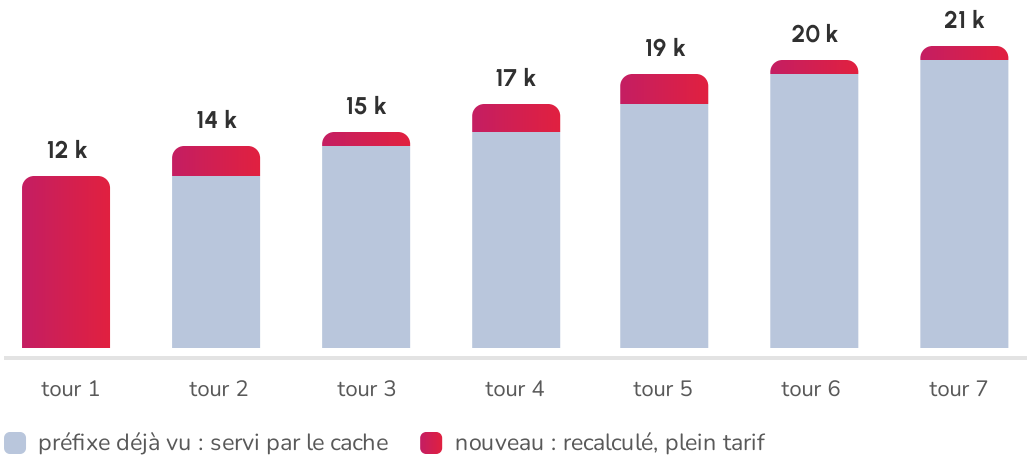
Le préfixe est **repayé plein tarif**, et la réponse arrive plus tard.

DURÉE DE VIE

De quelques minutes à quelques heures sans activité.

© ZENIKA 2026 | All rights reserved

Ce que coûte cette trace



118 k tokens d'entrée envoyés au total, dont 82 % lus en cache

LE CONSTAT

L'entrée fait le volume

Chaque tour renvoie tout le préfixe déjà vu, et n'ajoute que quelques milliers de tokens.

AVEC LE PROMPT CACHING

Ce préfixe est lu en cache, à ~10 % du prix.

LA SORTIE

~5× plus chère au token, mais petite en volume.

i Réduire ce qui **entre** rapporte plus que raccourcir la **sortie**.

© ZENIKA 2026 | All rights reserved

Tokens de raisonnement : choisir l'effort

LE PRINCIPE

Le modèle « réfléchit » en tokens avant de répondre

Ces **reasoning tokens** sont facturés comme de la **sortie**, et souvent invisibles.

×5

prix d'un token de sortie face à l'entrée



EFFORT BAS · MODÈLE LÉGER

Renommer, documenter, appliquer un pattern.



EFFORT MOYEN · GÉNÉRALISTE

Paginer un endpoint et ses tests, relire un diff.



EFFORT HAUT · RAISONNEMENT

Bug difficile, migration, architecture.

i L'effort se choisit **par tâche** : plus lent, plus cher, et **pas toujours meilleur**.

© ZENIKA 2026 | All rights reserved

Quiz : où part la facture ?

QUIZ 3 min

System prompt et tool definitions	Log de build de 3 000 lignes	Nouvelle demande	Tokens de raisonnement
stable : en cache	payé, puis renvoyé à chaque tour	plein tarif, mais petite	prix de sortie, invisibles

TIP Garde le début du contexte stable : c'est lui que le cache rentabilise.

© ZENIKA 2026 | All rights reserved

Module 2 · 45 min

Le harness, actif d’équipe

Quelles briques entourent le modèle, et laquelle utiliser pour quoi ?



© ZENIKA 2026 | All rights reserved

Les capacités attendues d’un harness



Sous-agents
Contexte isolé, et le droit d’écrire.



MCP standard
N’importe quel serveur, déclaré dans un fichier versionné.



Instructions, skills, hooks
Personnaliser sans toucher au cœur.



Modes de pilotage
Monter l’autonomie, borner les droits.



Scriptable
Un mode non interactif, pour la CI.

i **Plateforme** intégrée ou **composition** d’extensions : le harness est un **actif d’équipe versionné**. Sinon, c’est de l’**agentic debt**.

© ZENIKA 2026 | All rights reserved

Une allowlist n’est pas une sandbox

Allowlist

CE QU’ELLE FILTRE

Des motifs de commande

CONTOURNEMENT

npm test && curl …, un script qui en appelle un autre

FORME

Une liste dans la config du harness

Sandbox

CE QU’ELLE BORNE

Les effets : fichiers, réseau, droits

CONTOURNEMENT

Aucun par le prompt : l’agent ne sort pas du périmètre

FORME

Conteneur, VM, compte dédié, réseau coupé

Le seul périmètre réel est l’**isolation**.

© ZENIKA 2026 | All rights reserved

Les primitives du harness : qui décide ?



Instructions projet

AGENTS.md : règles stables, conventions, commandes.

QUI DÉCIDE

personne : toujours chargées



Skill

Savoir-faire réutilisable, chargé quand la tâche s'y prête.

QUI DÉCIDE

le modèle, sur sa description



Hook

Script lancé à un moment précis du cycle de vie.

QUI DÉCIDE

le harness, à chaque événement



Sous-agent

Travail à part, ne renvoie que son résultat.

QUI DÉCIDE

l'agent ou toi



MCP

Accès à un système externe.

QUI DÉCIDE

le modèle, à la demande

i Le critère qui range tout : **qui décide** qu'une primitive s'applique.

© ZENIKA 2026 | All rights reserved

Hooks : automatiser le cycle de vie

Des **scripts déterministes**, lancés par le harness, quoi que dise le modèle.

AVANT UN TOOL

Bloquer

git push --force, rm -rf, ou réécrire une commande verbeuse.

APRÈS UN TOOL

Formater

Le fichier écrit, ou journaliser l'appel.

FIN DE TOUR

Vérifier

Lancer les tests, relire le diff produit.

FIN DE SESSION

Archiver

La trace, ou proposer une mise à jour des notes.

i « Lance toujours les tests » dans les instructions est une **suggestion**. Le hook est une **garantie**.

© ZENIKA 2026 | All rights reserved

Tool calling : ce qui transite réellement

① Le harness déclare

```
{
  "name": "run_tests",
  "description":
    "Lance les tests",
  "input_schema": {
    "type": "object",
    "properties": {
      "path": {
        "type": "string"
      }
    }
  }
}
```

② Le modèle demande

```
{
  "type": "tool_use",
  "id": "call_01",
  "name": "run_tests",
  "input": {
    "path": "tests/bookings.tes
  }
}
```

③ Le harness répond

```
{
  "type": "tool_result",
  "tool_use_id": "call_01",
  "content":
    "FAIL bookings:42
     nextCursor is
     undefined"
}
```

i Le modèle choisit **sur la description** : chaque tool definition part **en tokens** à chaque appel.

© ZENIKA 2026 | All rights reserved

Ce que le harness injecte



SESSION AGENTIQUE · LE HARNESS INJECTE BIEN PLUS QUE VOS MESSAGES

Sorties de commandes

Un build, un npm install ou une stack trace peuvent dominer la context window s'ils ne sont pas filtrés.

i La question n'est pas « l'agent est-il intelligent ? », mais « **ce qu'il a injecté aide-t-il la tâche ?** »

© ZENIKA 2026 | All rights reserved

Quiz : quelle primitive ?

QUIZ 3 min

« Toujours passer par le client HTTP interne »

instructions

Interdire git push --force, quoi qu'il arrive

hook

Migrer le schéma, une fois par mois

skill

Explorer 40 fichiers sans polluer la session

sous-agent

© ZENIKA 2026 | All rights reserved

Exercice 1 — Garde-fou déterministe

OPTIONNEL 20 min ↗

Le concept

Ce qui doit **toujours** arriver est un **hook**, pas une consigne.

La consigne

1. Choisis un besoin du quiz : bloquer une commande, lancer les tests…
2. Mets-le en place **avec le mécanisme de hook de ton harness**.
3. Demande ensuite à ton agent de faire **le contraire**.

Réussi quand…

- l'action est **bloquée**, ou la vérification **lancée**, par le harness ;
- **même** quand on demande le contraire.

Débrief : qu'a tenté l'agent pour passer outre ?

TIP Ce qui doit toujours arriver est un hook, pas une consigne.

© ZENIKA 2026 | All rights reserved

Module 3 · 25 min

Ingénierie du contexte

Comment donner à l'agent le bon contexte, au bon moment, au bon coût ?



© ZENIKA 2026 | All rights reserved

Ce qui entre dans la context window

Élément	Entre quand ?	Ensuite
AGENTS.md, tool definitions	au démarrage	dans chaque appel, en cache
Historique	à chaque échange	grossit, compacté près de la limite
Fichiers lus, sorties de tools	à la demande	renvoyés jusqu'à la fin de la session
Skills	quand le modèle les juge utiles	nom et description d'abord, contenu ensuite

i Pour le savoir : **regarder** la jauge, **demander** à l'agent ce qu'il a lu, **élaguer** ce qui ne sert plus.

© ZENIKA 2026 | All rights reserved

KIT DE CODE

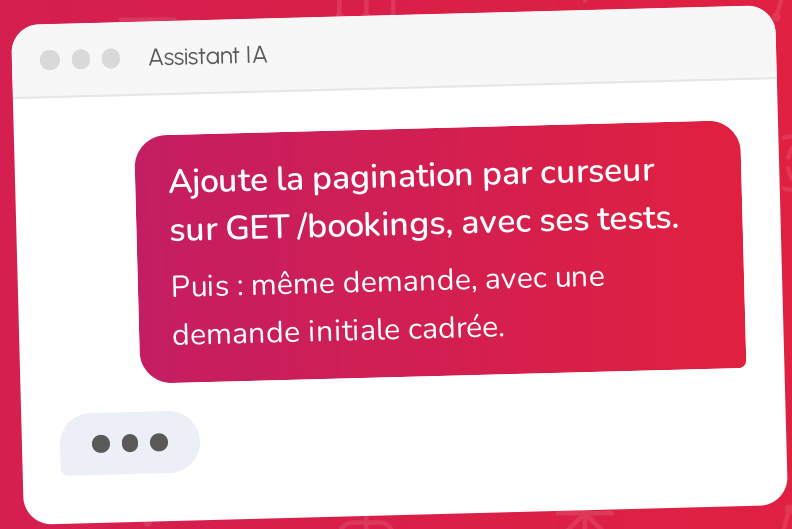
▶ DÉMO

Avec ou sans demande initiale

La même tâche, deux sessions : on compte les fichiers lus.

Demande initiale : objectif, fichiers clés, commandes, invariants, pièges.

Le gain annoncé n'est pas le gain mesuré : on compte.



© ZENIKA 2026 | All rights reserved

Réduire les tokens : un levier par buffer

SORTIES DE TOOLS

Filtrer

Erreurs seules, dernières lignes.

DONNÉES LOURDES

Traiter hors contexte

Un script, un index.

ENTRE SESSIONS

Mémoire ciblée

Ne plus tout réexpliquer.

RÉPONSE

Demander du concis

Pas de préambule.

CODE GÉNÉRÉ

En écrire moins

Moins à relire ensuite.

i Comprimer la prose du modèle rapporte peu : l'**entrée** domine. Faire **écrire moins de code** rapporte deux fois : maintenant et à la relecture.

© ZENIKA 2026 | All rights reserved

Le piège des sous-agents

Un cas vécu

Une demande de recherche un peu large

↓

6 sous-agents lancés en parallèle

↓

Chacun charge toutes les pages qu'il lit

↓

1 M de tokens en 30 min, quota épuisé

Ce qu'on retient

- La jauge de la session principale allait **très bien**.
- Elle ne montre pas ce que consomment les sous-agents.
- On borne **périmètre** et **nombre d'agents** avant de lancer.

i Un sous-agent protège ta context window, **pas ton budget**.

© ZENIKA 2026 | All rights reserved

La mémoire de l’agent

FICHIER VERSIONNÉ

Instructions et notes du projet

Relues en PR, comme du code.

MÉMOIRE DU HARNESS

L'agent écrit ses notes

Propriétaire, souvent plafonnée : à relire.

MIDDLEWARE TIERS

Serveur MCP ou plugin

Indexé, à votre main, exportable.

LE BON MÉCANISME : PROGRESSIVE DISCLOSURE

Chercher dans un index compact, puis ne charger que le détail des entrées retenues : ~10× moins de tokens.

i Une mémoire qu’on ne peut pas **relire** finit par contenir des faits périmés.

© ZENIKA 2026 | All rights reserved

Exploiter les traces

Une session contient

- ce qui a été **tenté** et ce qui a **échoué**
- ce qui **manquait** : doc, tool, convention
- ce qui a **fini par marcher**

C'est de l'**inférence déjà payée**.

Une extraction ciblée

Fichier	Contenu
`troubleshooting.md`	erreur → cause → fix
`api-fixes.md`	noms, docs à corriger
`next-session.md`	ce qu'on recharge

Non relue, une extraction n'est jamais une source d'autorité.

TIP Donne à l'agent de quoi chercher, pas tout le dépôt.

© ZENIKA 2026 | All rights reserved

Pause · 10 min

On souffle



© ZENIKA 2026 | All rights reserved

Module 4 · 45 min

Spec-driven et qualité

Comment garder le code aligné sur l'intention, et le prouver ?



© ZENIKA 2026 | All rights reserved

Ce que l'IA change dans notre façon de travailler

L'agent a compressé le temps de **production**, pas celui de **vérification**.

Hier	Avec un agent
RYTHME *Une feature = des jours*	**RYTHME** *Une feature = des heures*
CE QUI GARANTIT *Revues et rituels, a posteriori*	**CE QUI GARANTIT** *Contrôles systématiques, à chaque étape*

i Les revues **ne disparaissent pas** : elles ne suffisent plus **seules**. On leur adjoint des **contrôles automatiques**, au rythme de l'agent.

© ZENIKA 2026 | All rights reserved

Trois postures face à la spec

SDD : la spec versionnée est la source de vérité que l'agent implémente et que les tests vérifient.

1



Spec-first

Écrite d'abord, comme prompt structuré. Non maintenue : elle diverge du code.

2



Spec-anchored

Spec et code évoluent ensemble. Les écarts sont détectés.

3



Spec-as-source

Seule la spec est éditée. Le code devient un artefact généré.

i Cible réaliste : **spec-anchored**. L'agent sait **formaliser** une spec, pas décider **ce qui doit être vrai**.

© ZENIKA 2026 | All rights reserved

Spec-centric ou workflow-centric

	Spec-centric	Workflow-centric
Exemples	Spec Kit, OpenSpec, BMAD	Superpowers
Référence	la spec, versionnée	la discipline, encodée en skills
Ce qui survit	un document relisible	un process reproductible
Point fort	mémoire longue du projet	boucle d'implémentation serrée

i Pas un concurrent à départager : les deux se **combinent**.

© ZENIKA 2026 | All rights reserved

Garde-fous : du code au commit

L'agent génère vite : sans garde-fou, l'écart avec l'intention passe inaperçu.

1



Tests

Le contrat exécutable.

VÉRIFIE

le comportement

2



Revue agentique

Par un agent distinct de celui qui a écrit le code.

VÉRIFIE

le diff, avant d'envoyer

3



Commit

Le point de synchronisation entre code, spec et IA.

VÉRIFIE

la doc suit, la part d'IA est tracée

i **Règle minimale** : aucun code généré ne part en production sans **vérification déterministe** (tests, analyse statique) et **revue par un agent distinct**.

© ZENIKA 2026 | All rights reserved

Tests générés : deux natures, une mesure

TESTS DE CONSTRUCTION

Servent à l'agent

Testent l'implémentation, unité par unité. Jetables.

TESTS FONCTIONNELS

Servent à l'équipe

Testent les effets observables. **Durables** : la spec exécutable.

SCORE DE MUTATION

On modifie le code exprès (`<` devient `⩽`) : la suite doit échouer. Bien plus dur à tricher que la couverture.

i En fin de feature, là où les deux se recouvrent, **le test fonctionnel gagne**.

© ZENIKA 2026 | All rights reserved

Le drift spec ↔ code : qui le détecte ?

Drift : l'écart qui s'installe entre la spec et le code.

Détecteur	Ce qu'il voit	Ce qu'il rate
Tests fonctionnels	un comportement qui contredit une exigence encodée	une exigence nouvelle, sans test
Revue de spec dans la PR	un diff qui ne raconte pas la même chose que la spec	ce que le relecteur n'a pas lu
Contrôle au commit	du code changé sans spec touchée	une spec touchée… mais fausse
Personne	—	tout : le drift est silencieux

i Quand une exigence change, c'est la **revue de spec** qui le voit d'abord.

© ZENIKA 2026 | All rights reserved

Exercice 2 — La spec change

OPTIONNEL 25 min ↗

Le concept

La spec versionnée est la source de vérité ; le drift se détecte, il ne se devine pas.

La consigne

1. Écris une spec courte pour une fonctionnalité de l'API.
2. Versionne-la, fais-la implémenter avec ses tests.
3. En cours de route, change une exigence.

Réussi quand…

- la spec est versionnée dans le dépôt ;
- l'implémentation passe ses tests ;
- le changement est détecté, et on sait dire par quoi.

Débrief : qu'est-ce qui aurait laissé passer le drift ?

TIP Si la spec n'est pas versionnée avec le code, elle ment déjà.

© ZENIKA 2026 | All rights reserved

Module 5 · 30 min

Patterns agentiques

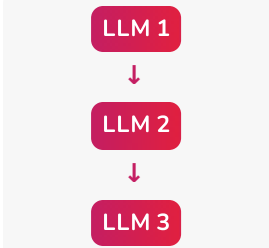
Qui décide de l'étape suivante, et qui juge le résultat ?



© ZENIKA 2026 | All rights reserved

Cinq patterns d'orchestration

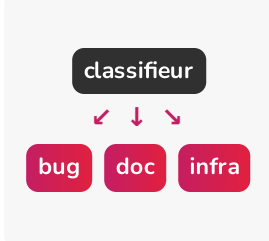
Chaîne



étapes fixes, chaque sortie nourrit la suivante

spec → plan → code

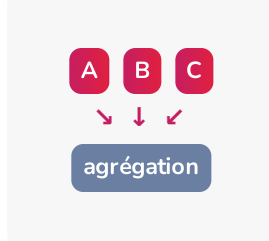
Routeur



on oriente vers le bon agent ou le bon modèle

un SLM trie, un LLM traite

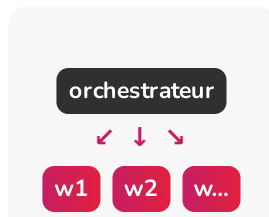
Parallélisation



tâches indépendantes, ou vote entre plusieurs

3 revues d'une même PR

Orchestrateur-workers



le découpage est décidé dynamiquement par un LLM

refacto sur 12 modules

Évaluateur



on itère jusqu'à ce que le critère soit atteint

code + revue par un 2e agent

workflow figé : chemin connu à l'avance — *workflow dynamique : le LLM pilote davantage*

Cinq **workflows** : le **code** décide de l'étape suivante.

© ZENIKA 2026 | All rights reserved

Patterns agentiques : qui décide, qui juge ?

REACT

Penser, agir, observer

Décide : le modèle, à chaque tour.
Juge : les sorties des tools.

PLAN-AND-EXECUTE

Planifier, puis dérouler

Décide : le plan, écrit par le modèle.
Juge : une vérification par étape.

GOAP

Viser un état but

Décide : un planificateur, sur des actions déclarées.
Juge : l'état but atteint.

REFLECTION

Se relire

Décide : le modèle.
Juge : le même modèle, avec ses angles morts.

AGENT AS A JUDGE

Un autre agent évalue

Décide : le générateur.
Juge : un agent distinct, avec une grille.

BEST-OF-N

N tentatives, une gagnante

Décide : N agents isolés.
Juge : les tests, puis un juge.

i Plus le **modèle** décide, plus il faut un **juge** solide.

© ZENIKA 2026 | All rights reserved

Plan-and-Execute ou GOAP : qui écrit le plan ?

	Plan-and-Execute	GOAP
Qui planifie	le LLM, en langage naturel	un algorithme, sur des actions déclarées
Le plan	souple, mais une étape peut être inventée	chaque action a ses préconditions et ses effets : test → code → contrat → PR
Imprévu	on corrige le plan, ou on relance	il replanifie depuis l'état réel
Pour	tâches ouvertes : explorer, refactorer	processus connus : livrer, migrer, déployer

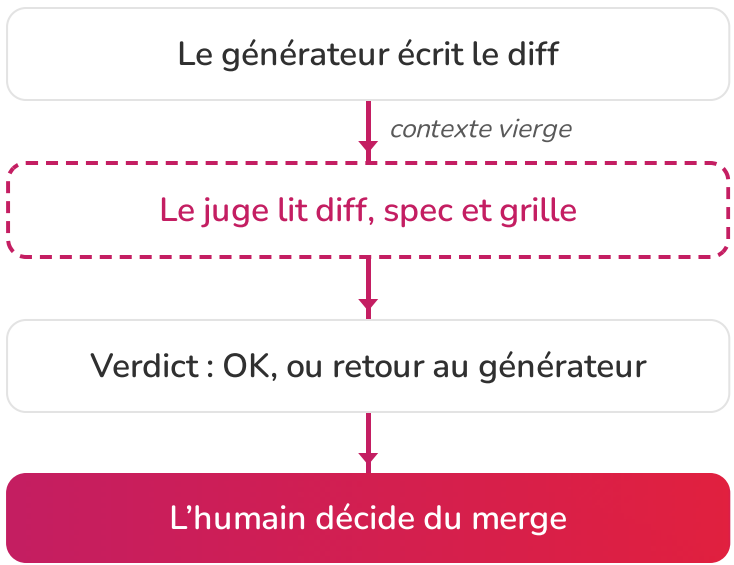


GOAP : le LLM **exécute** les actions, il ne **choisit** plus leur ordre. Prévisible, explicable.

© ZENIKA 2026 | All rights reserved

Agent as a judge : une grille, un autre contexte

Générer, juger, décider



UNE GRILLE

Des critères écrits : la grille « Relire le code généré » de la Foundation.

UN AUTRE CONTEXTE

Il n'a pas vu la session : pas de complaisance. Idéalement, un autre modèle.

DES BIAIS CONNUS

Il préfère sa propre prose, les réponses longues, la première option : calibrer sur des cas jugés par un humain.

i Le juge **prépare la preuve** ; l'humain **décide**.

© ZENIKA 2026 | All rights reserved

Best-of-N : N tentatives, un juge

LE PRINCIPE

La même tâche, lancée N fois en parallèle

Chaque tentative est **isolée**. Les tests filtrent, un juge départage ce qui reste.

× N

tokens, et autant de diffs à départager

QUAND

Une tâche incertaine, plusieurs approches plausibles : un bug coriace, un algorithme.

ISOLER

Une tentative par worktree : elles ne se marchent pas dessus (module 6).

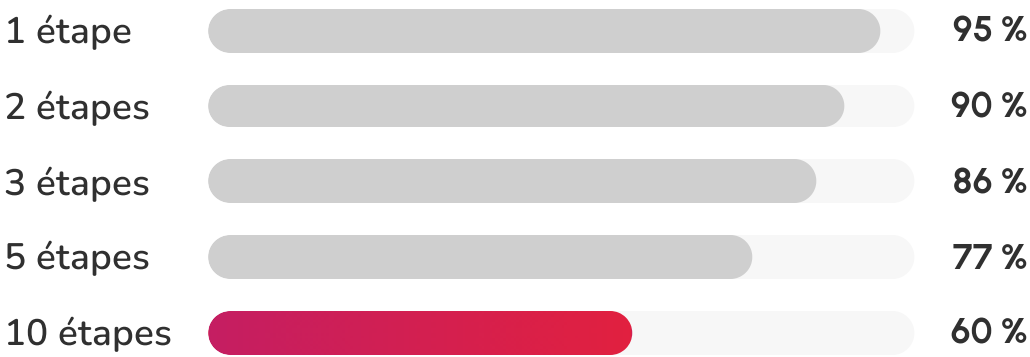
JUGER

D'abord les tests, déterministes ; puis le juge sur les survivantes.

i Best-of-N achète de la **fiabilité** avec des **tokens**. Sans juge, c'est **vous** qui départagez.

© ZENIKA 2026 | All rights reserved

La fiabilité composée : 10 étapes à 95 % ≈ 60 %



Fiabilité d'une chaîne d'agents fiables à 95 % chacun

LE CALCUL

Les fiabilités se multiplient

0,95 × 0,95 × … dix fois ≈ **0,60**.

LA PARADE

Une chaîne **courte**, et un **juge** entre les étapes : les tests, puis un agent.

© ZENIKA 2026 | All rights reserved

Le coût caché : l'érosion de la connaissance



Écrire moins

Le code vous appartient moins quand vous ne l'écrivez pas.



Relire n'est pas apprendre

On valide une forme, pas un raisonnement.



Lent et invisible

On le découvre en incident, deux ans plus tard.

i Ce n'est pas un argument contre l'autonomie, c'est sa **condition** : ce qu'on automatise, il faut savoir le **reprendre**.

© ZENIKA 2026 | All rights reserved

Quiz : quel pattern ?

QUIZ 3 min

« Propose un plan, je le relis, puis exécute-le étape par étape »

Plan-and-Execute

Test, code, contrat, PR : un planificateur choisit l'ordre et replanifie si un test casse

GOAP

Un agent qui n'a pas vu la session relit le diff avec la grille

agent as a judge

Trois agents tentent le même bug ; on garde celui dont les tests passent

best-of-N

TIP Ajoute un agent seulement si tu sais qui jugera son travail.

© ZENIKA 2026 | All rights reserved

Module 6 · 40 min

Paralléliser : worktrees et ADE

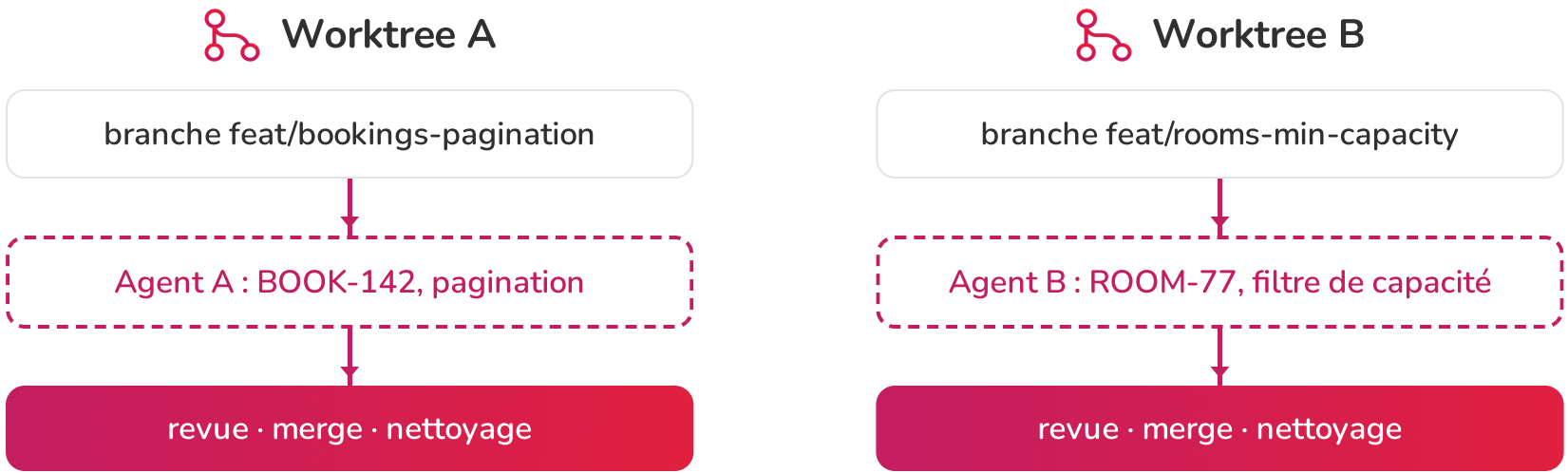
Plusieurs agents en même temps, sans tout casser.



© ZENIKA 2026 | All rights reserved

Agents en parallèle sur worktrees

Un dépôt, **deux répertoires de travail**, deux agents **en même temps**.



Paralléliser demande d'isoler : chaque agent a ses fichiers, sa branche et son contexte.

© ZENIKA 2026 | All rights reserved

L’ADE : piloter des agents, plus des fichiers

IDE	ADE · Agentic Development Environment
UNITÉ DE TRAVAIL *Le fichier, l’onglet*	**UNITÉ DE TRAVAIL** *La session d’agent : un worktree, une branche* *Conductor, Orca*
CE QU’ON REGARDE *Le code qu’on écrit*	**CE QU’ON REGARDE** *N agents : état, terminal, diff*
CE QU’ON VALIDE *Sa propre ligne*	**CE QU’ON VALIDE** *Un diff, puis une PR*
OÙ TOURNE L’AGENT *Dans l’éditeur*	**OÙ TOURNE L’AGENT** *Sur le poste, en arrière-plan ou dans le cloud*

i L’éditeur reste, comme **un outil parmi d’autres**. Retenez le **rôle**, pas le produit.

© ZENIKA 2026 | All rights reserved

Développement parallèle : le bilan

Ce qu'on gagne

TEMPS
Deux features avancent ensemble

ISOLATION
Les agents ne se perturbent pas

REVUE
Des PR plus petites, plus lisibles

SUPERVISION
Un ADE montre tout au même endroit

Ce que ça coûte

TOKENS × N
Chaque agent a son propre contexte

CONFLITS DE MERGE
Si les features touchent les mêmes fichiers

SERVICES PARTAGÉS
Ports, base de données, dépendances

RELECTURE
Tout passe par vous : le goulot, c'est l'humain

TIP Features indépendantes seulement, et quelqu'un pour relire : au-delà de 2 sessions, on subit.

© ZENIKA 2026 | All rights reserved

De la revue locale à la revue de PR en CI

Revue locale

EXÉCUTION
Un hook sur le poste

PÉRIMÈTRE VU
Diff local et conventions

RESTITUTION
Dans la session

Revue de PR en CI

EXÉCUTION
Le runner CI, à chaque PR

PÉRIMÈTRE VU
PR complète, spec, règles d'équipe, analyse statique

RESTITUTION
Commentaire de PR et quality gate

i Plus d'agents, plus de PR : la **CI relit** d'abord, puis le merge suit le **risque**.

© ZENIKA 2026 | All rights reserved

Exercice 3 — Deux agents, deux worktrees

OPTIONNEL 20 min ↗

Le concept

Paralléliser demande d'**isoler** : un worktree, une branche, un agent.

La consigne

1. Crée **deux worktrees** du kit, avec git ou ton ADE : un pour `TICKET.md`, un pour `tickets/ROOM-77.md`.
2. Lance **un agent dans chacun**, en même temps.
3. **Merge** les deux branches.

Réussi quand…

- chaque branche a ses **tests verts** ;
- les deux sont **mergées**, et `npm test` passe ;
- tu sais dire ce qui aurait pu **entrer en conflit**.

i **Débrief** : combien de sessions pouviez-vous vraiment suivre ?

© ZENIKA 2026 | All rights reserved

Clôture · 10 min

Ce qu’on garde



© ZENIKA 2026 | All rights reserved

Une requête, de bout en bout



Une tâche issue
d'une **spec relue**.

1. Spec

2. Pattern



Un **plan**,
un **juge** prévu.



Un agent **isolé**,
sur sa branche.

3. Worktree

4. Harness



Contexte **stable**,
hooks et permissions.



Les **tests**, puis
un **agent distinct**.

5. Juge

6. PR



La **CI** relit,
l'**humain** décide.

i **Un dev, plusieurs agents, un système** : configurer, isoler, juger.

© ZENIKA 2026 | All rights reserved

Trois idées à garder

1



Le contexte fait la facture

L'entrée domine, le préfixe stable est en cache, les sous-agents isolent sans être gratuits.

RÉFLEXE

Mesurer ce qui entre.

2



Ce qui doit arriver se configure

Hook, sandbox, tests, spec versionnée : ça ne se demande pas au modèle.

RÉFLEXE

Hook plutôt que consigne.

3



Plusieurs agents : isoler et juger

Un worktree par agent, un juge entre les étapes, un humain avant le merge.

RÉFLEXE

Pas d'agent sans juge.

© ZENIKA 2026 | All rights reserved

Quiz de sortie

QUIZ 3 min

1

Pourquoi brancher un MCP en cours de session coûte-t-il cher ?

le cache saute : tout est repayé

2

« Lance toujours les tests » : où l'écrire pour que ce soit garanti ?

dans un hook

3

Des actions avec préconditions et effets, ordonnées par un algorithme : quel pattern ?

GOAP

4

Quatre agents en parallèle : qu'est-ce qui sature d'abord ?

vous : la relecture

© ZENIKA 2026 | All rights reserved

Questions

Un dev, plusieurs agents, un système.

© ZENIKA 2026 | All rights reserved